

**BRACT's**

**Vishwakarma Institute of Technology, Pune**

**Practical Implementation Sheet**

Department: CSE-AI      Semester: 5      Academic Year: 2026-27

Division: E      Batch: 1      Practical No: 2

Roll no: 08      PRN No: 12412097      Name of Student: Sahil Balasaheb Patil

**Subject Name : Deep Learning**

### **Problem Statement :**

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

### **Algorithm :**

- 1) Start the program.
- 2) Import the required libraries:
  - i. Numpy
  - ii. Matplotlib
  - iii. Pandas
  - iv. Seaborn
  - v. Scikit-learn
  - vi. TensorFlow/Keras
- 3) Load the Iris dataset (Iris.csv) into a Pandas DataFrame. Separate the dataset into:
  - i. Training data
  - ii. Testing data
- 4) Display the dataset information such as:
  - i. Shape of the dataset
  - ii. Column names
  - iii. First five records

5) Preprocess the dataset:

- i. Remove the Id column (if present).
- ii. Separate the input features (X) and target labels (y).
- iii. Convert categorical target labels into numerical values using LabelEncoder.

6) Split the dataset into training and testing sets using an 80:20 ratio.

7) Normalize the input features using StandardScaler.

8) Train the MLP model using the training dataset for 50 epochs with a batch size of 8.

9) Evaluate the trained model using the testing dataset.

10) Visualize the Confusion Matrix using a heatmap.

## Code :

```
3] from sklearn.datasets import load_iris
   from sklearn.model_selection import train_test_split
   from sklearn.preprocessing import StandardScaler
   from sklearn.neural_network import MLPClassifier
   from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

4] iris = load_iris()
   X = iris.data
   y = iris.target

5] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

6] scaler = StandardScaler()
   X_train = scaler.fit_transform(X_train)
   X_test = scaler.transform(X_test)

8] mlp = MLPClassifier(
      hidden_layer_sizes=(10, 10),
      activation='relu',
      solver='adam',
      max_iter=1000,
      random_state=42
```

```
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

9] print("Accuracy:", accuracy)
   print("\nConfusion Matrix:")
   print(cm)

... Accuracy: 0.9666666666666667

Confusion Matrix:
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

10] print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=iris.target_names))

Classification Report:
              precision    recall  f1-score   support

   setosa       1.00      1.00      1.00        10
  versicolor    1.00      0.89      0.94         9
   virginica    0.92      1.00      0.96        11

   accuracy                   0.97        30
  macro avg       0.97      0.96      0.97        30
```





**Output :**



